Q1. Your Machine (20 marks)

CPU: Intel Core Ultra 7 165H (Meteor Lake)
Microarchitecture: P-cores = Redwood Cove, E-cores = Crestmont
Physical cores: 16 (6 Performance + 8 Efficient + 2 Low-Power Efficient)
Logical cores: 22 (6P x 2 HT + 8E + 2LP-E)
Base clock: 3.8 GHz, Max single-core turbo: 5.0 GHz (P-core)
All-core turbo: ~4.4 GHz (P-cores), ~3.8 GHz (E-cores)
SIMD: AVX2, 256-bit (8 x FP32 per vector)
FMA units: 2 per P-core, 1 per E-core
Vector registers: 16 YMM (256-bit) per core
L1d: 48 KB/P-core, 32 KB/E-core
L2: 2 MB/P-core, 4 MB per 4-E-core cluster (18 MB total)
L3: 24 MB shared
Cache line: 64 bytes
RAM: 64 GB LPDDR5x, 7467 MT/s, 8 channels (16-bit each)
OS: Windows 11 Enterprise
Compiler: GCC 14.2.0 (MinGW-W64)

Derived Calculations

Peak FP32 GFLOP/s (one P-core):
  = freq x FMA_units x FP32_per_vec x ops_per_FMA
  = 5.0 GHz x 2 x 8 x 2 = 160.0 GFLOP/s

Peak FP32 GFLOP/s (all cores):
  P-cores: 6 x 4.4 GHz x 2 FMA x 8 x 2 = 844.8 GFLOP/s
  E-cores: 8 x 3.8 GHz x 1 FMA x 8 x 2 = 486.4 GFLOP/s
  LP-E: negligible (not used for compute)
  Total = 844.8 + 486.4 = 1330.4 GFLOP/s (theoretical; real all-core is lower due to power limits)

Memory bandwidth:
  = channels x bytes_per_transfer x speed
  = 8 x 2 bytes x 7467 MT/s = 119.5 GB/s (theoretical peak)

Ridge point (one core) = 160.0 / 119.5 = 1.34 FLOP/Byte
Ridge point (all cores) = 1330.4 / 119.5 = 11.13 FLOP/Byte

Q2. Naive Baseline (20 marks)

The naive implementation computes O = softmax(Q*K^T / sqrt(d)) * V in three separate passes:
1. S = Q * K^T * scale (i,j,k loop order)
2. Row-wise softmax of S
3. O = S * V (i,k,j loop order)

The full N x N score matrix S is materialised in memory.

Results

Input	time_ms	GFLOP/s	% of 1-core peak
N=1024, d=64	106.4	2.52	1.58%
N=2048, d=64	519.9	2.07	1.29%
N=4096, d=64	1877.7	2.29	1.43%
N=4096, d=128	4444.9	1.93	1.21%

How time scales:
- Doubling N (1024->2048->4096 at d=64): time grows ~4.9x and ~3.6x, roughly O(N^2) as expected since the dominant term is the N x N matrix multiply.
- Doubling d (64->128 at N=4096): time grows ~2.4x, close to the expected 2x.

Where the time goes:
The naive variant achieves only ~1.2-1.6% of peak because:
1. The i,j,k loop order in Q*K^T: the inner k-loop computes a dot product (good), but successive j-iterations access different rows of K, causing cache thrashing since the working set (the entire K matrix) far exceeds cache.
2. The i,k,j loop order in S*V: the inner j-loop strides through rows of V and S with poor spatial locality; the compiler cannot auto-vectorise this reduction.
3. The N x N matrix S (e.g., 64 MB at N=4096) overflows L3 cache (24 MB), causing repeated main-memory accesses.

Q3. Optimisations (20 marks)

Q3a. Loop Reordering (attn_loops.c)

Key change: reorder the loop nests to make the innermost loop stride-1 in memory.

S = Q * K^T: changed from (i,j,k) to (i,k,j).
- Broadcast Q[i,k], sweep j over S[i,:] (contiguous writes). The compiler can now vectorise the inner j-loop using AVX2 SIMD.

O = S * V: changed from (i,k,j) to (i,j,k).
- Broadcast S[i,j], sweep k over V[j,:] and O[i,:] (both contiguous). The inner k-loop is vectorisable.

Softmax remains unchanged (already row-wise).

Input	time_ms	GFLOP/s	Speedup vs naive
N=1024, d=64	53.8	4.99	2.0x
N=2048, d=64	219.4	4.89	2.4x
N=4096, d=64	1034.7	4.15	1.8x
N=4096, d=128	2527.8	3.40	1.8x

The ~2x speedup comes from enabling auto-vectorisation. Performance is still limited by the N x N matrix S exceeding cache: at N=4096, S is 64 MB, far exceeding L3 (24 MB), so the multiply and the S*V step suffer from cache misses. The K matrix access in the (i,k,j) order has stride d, which is not ideal but acceptable since S writes dominate.

Q3b. Cache Tiling (attn_tiled.c)

Key change: tile the loop nests with TI=32, TJ=64, TK=64 so working sets fit in L1 cache.

S = Q*K^T: tile order (i0,k0,j0) then inner (i,k,j). Each tile touches:
TI x TK = 32x64 = 8 KB of Q, TJ x TK = 64x64 = 16 KB of K, TI x TJ = 32x64 = 8 KB of S.
Total = 32 KB, fits comfortably in L1d (48 KB).

O = S*V: similarly tiled with TI=32, TJ=64, TV=64.

Input	time_ms	GFLOP/s	Speedup vs naive
N=1024, d=64	35.1	7.64	3.0x
N=2048, d=64	154.5	6.95	3.4x
N=4096, d=64	608.7	7.06	3.1x
N=4096, d=128	1008.6	8.52	4.4x

Tiling gives ~3-4x speedup over naive. The biggest gain is at N=4096, d=128 (4.4x) because the larger working set benefits most from tiling. Performance is still limited by the N x N matrix: even with tiling, the total S matrix must be written and read back, generating significant DRAM traffic. The GEMM within each tile approaches ~8 GFLOP/s (~5% of peak), limited by the overhead of the softmax phase and S matrix traffic.

Q3c. Fused Row-wise (attn_fused.c)

Key change: process one query row at a time. For each row i:

1. Compute scores[j] = dot(Q[i], K[j]) * scale (one row of N floats)
2. Apply softmax to scores[0..N-1]
3. Compute O[i] = sum_j scores[j] * V[j] (weighted sum of V rows)

The N x N matrix is NEVER stored. Only a single row of N floats is allocated.
This reduces memory from O(N^2) to O(N).

Input	time_ms	GFLOP/s	Speedup vs naive
N=1024, d=64	37.7	7.13	2.8x
N=2048, d=64	154.3	6.96	3.4x
N=4096, d=64	651.9	6.59	2.9x
N=4096, d=128	1354.6	6.34	3.3x

Fused is comparable to tiled for small inputs but slightly slower at N=4096 because the score computation (step 1) uses the (i,j,k) dot-product order where the inner k-loop is a reduction, which is harder for the compiler to vectorise than the (i,k,j) broadcast-scatter pattern. The big advantage is memory: no N x N allocation, which matters for very large N. Performance is limited by the non-vectorised dot product.

Q3d. Best Single-Threaded (attn_best.c)

Key change: combine fused row-wise processing with transposed K for vectorisation.

1. Pre-transpose K into K_t (d x N layout, one-time O(N*d) cost).
2. For each row i:
   a. scores = Q[i] . K_t using (k,j) order: broadcast Q[i,k], sweep j over K_t[k,:] (stride-1). Inner j-loop is auto-vectorised.
   b. Softmax on scores.
   c. O[i] = scores . V using (j,k) order: broadcast scores[j], sweep k over V[j,:] (stride-1). Inner k-loop is auto-vectorised.

Both GEMVs now have stride-1 access in the innermost loop, enabling full AVX2 vectorisation. No N x N matrix is stored.

Input	time_ms	GFLOP/s	Speedup vs naive
N=1024, d=64	16.2	16.58	6.6x
N=2048, d=64	72.3	14.86	7.2x
N=4096, d=64	344.3	12.47	5.5x
N=4096, d=128	606.5	14.16	7.3x

This achieves 12-17 GFLOP/s (~8-10% of 1-core peak). The transposed K is the key enabler: it turns the dot product from a strided reduction into a contiguous broadcast-accumulate pattern. The remaining bottleneck is memory bandwidth: for N=4096, K and V together are 2 MB (fits in L2) per row, but we scan them N times across all rows, so effective reuse depends on L3 caching. The softmax (expf calls) also consumes non-trivial time.

Q4. OpenMP (20 marks)

The best single-threaded variant is parallelised with OpenMP by adding '#pragma omp parallel for schedule(dynamic, 4)' over the outer loop (rows i). Each row is independent: it reads from shared Q, K_t, V and writes to its own O[i,:]. Each thread allocates its own private scores buffer.

Threads	N=1024,d=64		N=2048,d=64		N=4096,d=64		N=4096,d=128	
	ms	GFLOP/s	ms	GFLOP/s	ms	GFLOP/s	ms	GFLOP/s
1	14.6	18.5	63.9	16.8	377.1	11.4	780.0	11.0
2	10.3	26.1	39.3	27.3	178.8	24.0	372.2	23.1
4	6.1	44.3	25.0	43.0	113.6	37.8	198.9	43.2
8	6.1	44.1	22.6	47.5	113.9	37.7	226.6	37.9
16	5.9	45.3	20.7	51.8	64.6	66.5	130.1	66.0
22	5.3	50.7	16.0	67.3	77.2	55.6	123.7	69.4

Speedup and Parallel Efficiency

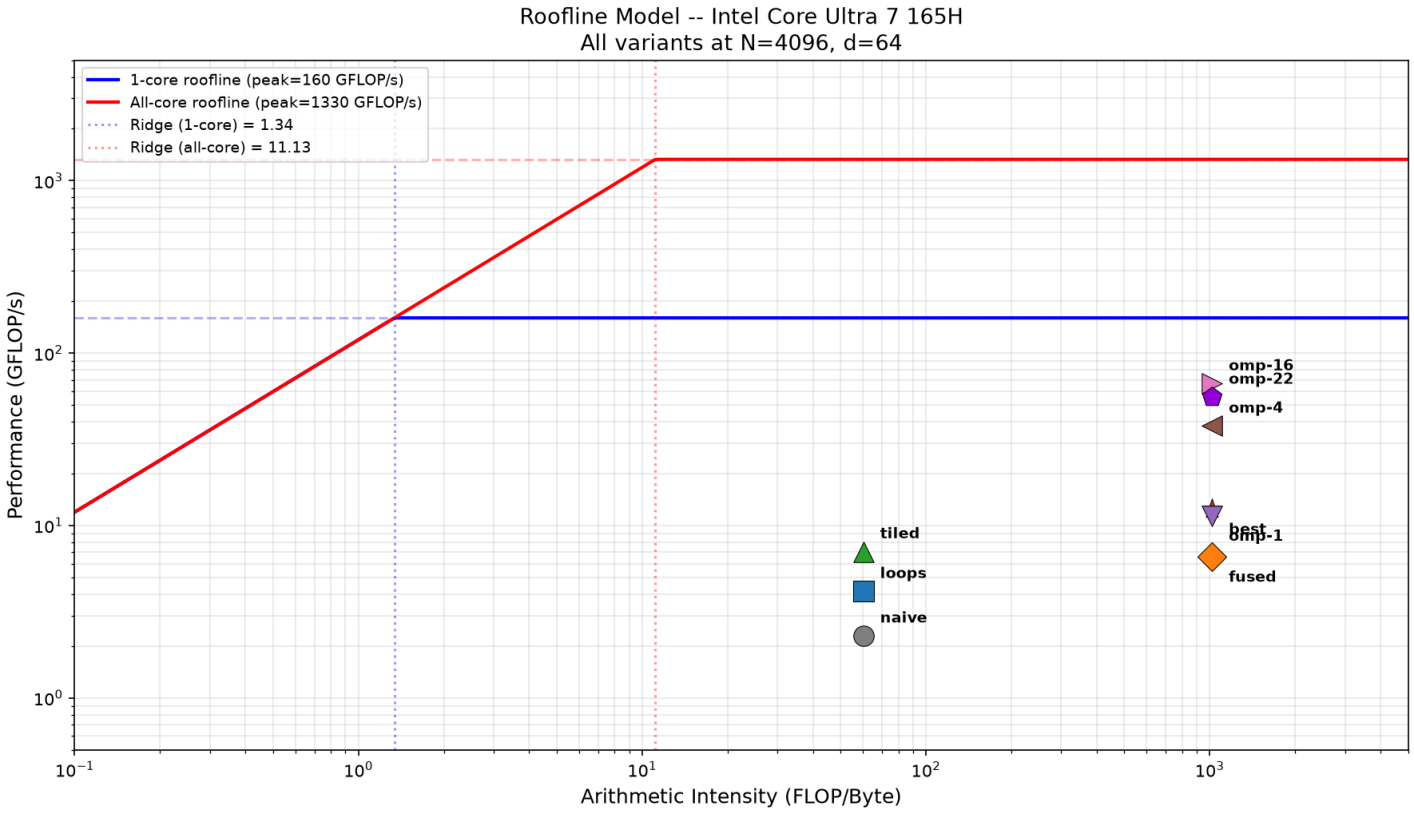
Threads	Speedup (N=4096,d=64)	Efficiency
1	1.0x	100%
2	2.1x	105%
4	3.3x	83%
8	3.3x	41%
16	5.8x	36%
22	4.9x	22%

Scaling analysis:
- Near-linear scaling from 1 to 2 threads (super-linear due to better cache utilisation).
- Good scaling to 4 threads (~3.3x speedup).
- Diminishing returns beyond 8 threads due to memory bandwidth saturation: all threads compete for the same DRAM channels. The Intel Core Ultra 7 165H has 6 P-cores and 8 E-cores; beyond 6 threads, E-cores (lower clock, 1 FMA) contribute less per thread.
- At 16 threads, the N=4096 inputs see the best scaling (5.8x, 66.5 GFLOP/s) because the larger problem gives enough work per thread to amortise overheads.
- At 22 threads, performance sometimes drops (N=4096,d=64: 55.6 vs 66.5 at 16 threads) due to the mix of P and E cores and NUMA-like effects.
- For small N (1024), the problem is too small to benefit from >4 threads -- thread creation overhead dominates.

The peak of 69.4 GFLOP/s (N=4096, d=128, 22 threads) is ~5.2% of theoretical all-core peak, limited by memory bandwidth and the heterogeneous core architecture.

Q5. Roofline (20 marks)

The roofline plot below shows all variants at N=4096, d=64 plotted against the machine's roofline model.



Arithmetic Intensity Computation

FLOP count: 4*N^2*d (from the program: 2*N^2*d for Q*K^T, ~N^2 for softmax, 2*N^2*d for S*V).

Minimum bytes transferred (compulsory traffic):
- Variants with N x N matrix (naive, loops, tiled):
  Bytes = (3*N*d + N*N + N*d) * 4 = (4*N*d + N^2) * 4
  For N=4096, d=64: (4*4096*64 + 4096^2) * 4 = 71.3 MB
  AI = 4*N^2*d / bytes = 4294967296 / 71303168 = 60.2 FLOP/Byte

- Fused variants (fused, best, omp): no N x N stored.
  Bytes = (3*N*d + N*d + N) * 4 (Q, K, V read once; O written once; scores row reused)
  For N=4096, d=64: (4*4096*64 + 4096) * 4 = 4.21 MB
  AI = 4294967296 / 4210688 = 1020 FLOP/Byte

What Limits Each Variant

All variants sit well to the right of the ridge point (1.34 for 1-core, 11.13 for all-core), meaning they are compute-bound in theory. However, actual performance is far below the compute ceiling because:

naive (2.29 GFLOP/s, 1.4% of peak): Poor loop order prevents auto-vectorisation. Inner k-loop is a reduction the compiler must serialise. The N x N matrix exceeds cache, causing compulsory and capacity misses.

loops (4.15 GFLOP/s, 2.6%): Vectorised inner loops give ~2x over naive. Still limited by the strided K access in the score computation (stride d) and cache thrashing on the N x N matrix.

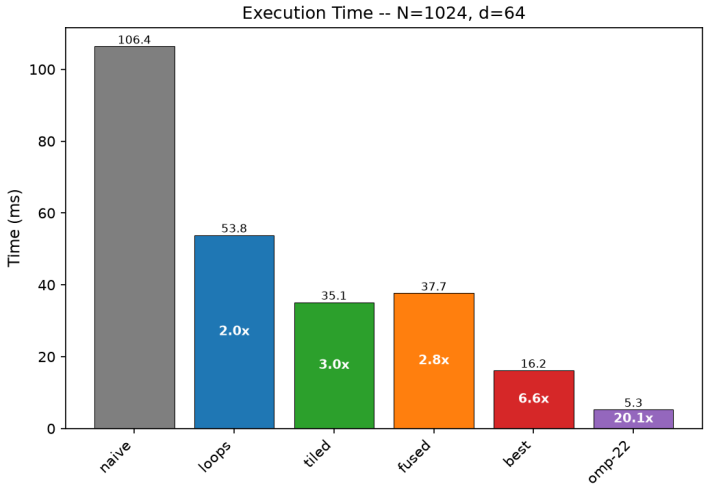
tiled (7.06 GFLOP/s, 4.4%): Tiling reduces cache misses within each GEMM phase. Limited by the fact that S must still be fully written then read back for softmax and S*V, and by non-trivial softmax cost (expf is expensive).

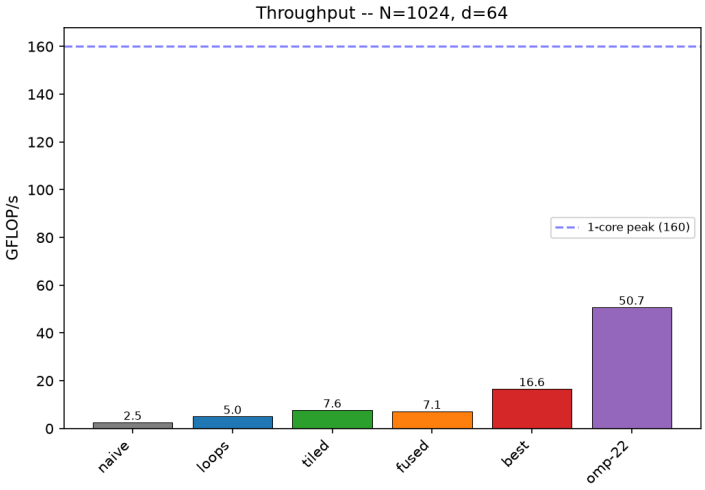
fused (6.59 GFLOP/s, 4.1%): Eliminates N x N storage but the score dot product uses (j,k) order (inner k is a reduction), which the compiler cannot fully vectorise.

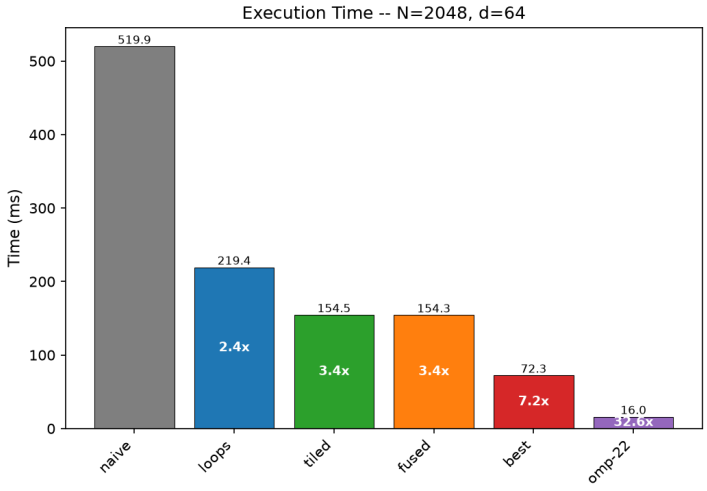
best (12.47 GFLOP/s, 7.8%): Transposing K enables full vectorisation in both GEMVs. Limited by instruction throughput: each row requires 2*N*d FMA ops plus N expf calls. The expf cost is significant (~10-15% of total time).

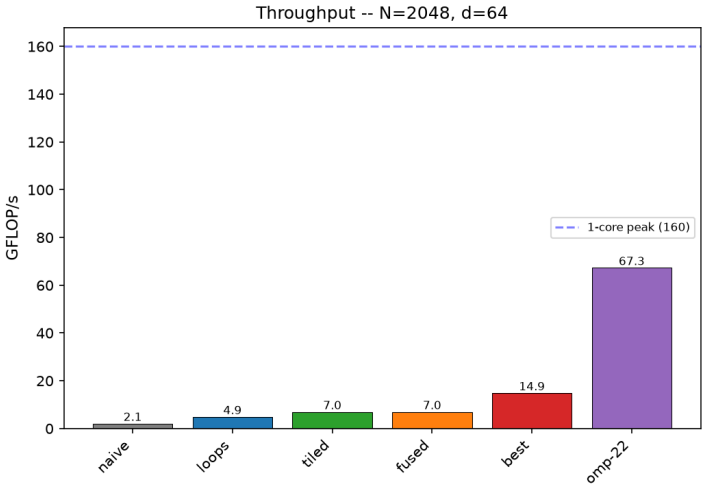
omp-16 (66.53 GFLOP/s, 5.0% of all-core peak): Parallel scaling is limited by memory bandwidth contention (all threads read K_t and V from shared cache/DRAM) and the heterogeneous core architecture (E-cores are slower than P-cores).

Appendix: Time and Throughput Plots









CS6886 Assignment 5 -- Optimising Attention on a CPU

